NAAN MUDHALVAN

Project Documentation

Implement Client Script & UI Policy (Incident)

ServiceNow micro project

Team ID	SWTID-2026-4524
Team Leader	Swathi R
Team Member	Sandhiya D
Team Member	Sandhya Charlet Calista S
Team Member	Mallemkonda Dallyjasmine

This documentation follows the standard project documentation format. Sections written for a MERN stack (React, Node.js, Express.js, MongoDB) are adapted to the ServiceNow platform, which this project uses.

1. Introduction

Project Title: Implement Client Script & UI Policy (Incident)

Team Members:

Name	Role	Team ID
Swathi R	Team Leader	SWTID-2026-4524
Sandhiya D	Team Member	SWTID-2026-4524
Sandhya Charlet Calista S	Team Member	SWTID-2026-4524
Mallemkonda Dallyjasmine	Team Member	SWTID-2026-4524

2. Project Overview

Purpose: Incident records need consistent and accurate data for effective triage, routing and resolution. The purpose of this project is to show how ServiceNow client-side controls (UI Policies and Client Scripts) enforce data integrity on the Incident form. The goals are to make fields mandatory or read-only based on Impact, fill in values automatically, and stop records from being submitted when required conditions are not met.

Features:

- **High Impact Control UI Policy:** applies when Impact is 1 - High and makes Assignment group mandatory; Reverse if false restores the fields when the condition is no longer met.
- **Read-only Urgency:** a UI Policy Action makes Urgency read-only while Impact is High.
- **Auto set Urgency (onChange):** Urgency is set to High and an info message is shown when Impact changes to High.
- **Save validation (onSubmit):** the save is blocked and an error is shown on Assigned To when Impact is High and Assigned To is empty.
- **List edit control (onCellEdit):** State cannot be changed directly in the list view; an alert asks the user to open the Incident.

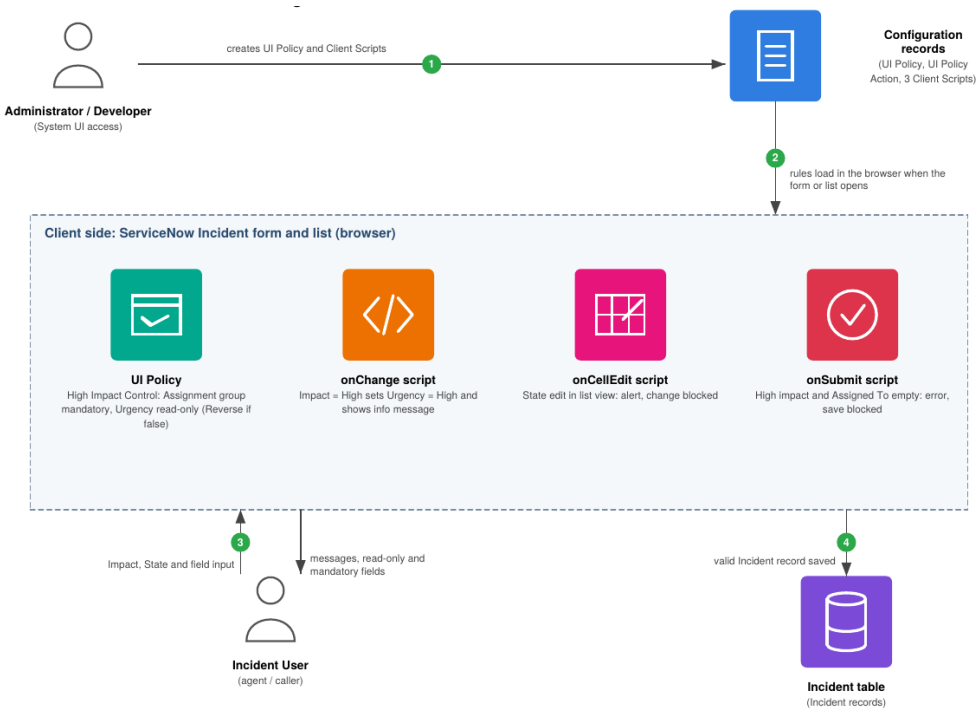
3. Architecture

This project runs on the ServiceNow platform instead of a MERN stack, so the three layers of the template are described as follows.

Frontend: The frontend is the ServiceNow Incident form and list in the web browser. The UI Policy, the UI Policy Action and the three Client Scripts (onChange, onSubmit, onCellEdit) run in the browser and control how fields behave. They use the g_form client API. React is not used.

Backend: The backend is the ServiceNow instance itself. It stores the configuration records (the UI Policy, its actions and the Client Scripts), delivers them to the browser when the form or list opens, and saves valid Incident records. Node.js and Express.js are not used, and no custom server code was written.

Database: Data is held in the ServiceNow platform. The Incident table stores Incident records, with the fields used by this project: Impact, Urgency, Assignment group, Assigned to and State. MongoDB is not used.



Flow 1: the Administrator or Developer creates the UI Policy, UI Policy Action and Client Scripts in System UI.

Flow 2: the rules load in the browser when an Incident form or list is opened.

Flow 3: the user changes Impact or State; the UI Policy, onChange and onCellEdit scripts respond and show messages.

Flow 4: on save, the onSubmit script checks Assigned To; a valid record is stored in the Incident table.

Figure 3.1: Solution architecture and data flow

4. Setup Instructions

Prerequisites:

- A ServiceNow instance with administrative or configuration access.
- A web browser to open the instance.
- The Incident application available on the instance. No Node.js, MongoDB or local installation is needed.

Installation (configuration steps):

Step	Action
1	Log in to the ServiceNow instance with administrative or configuration access.
2	Go to System UI > UI Policies and click New. Create the UI Policy "High Impact Control" on the Incident table, Active = true, condition Impact is 1 - High, make Assignment group mandatory and set Reverse if false to true. Click Submit.
3	Open the UI Policy, go to the UI Policy Actions related list and create an action for the Urgency field with Read-only = true (leave Visible unchanged). Click Submit.
4	Go to System UI > Client Scripts and click New. Create the onChange script "Auto set urgency for high impact" on Incident, field Impact, Active = true. Paste the script from the Appendix and click Submit.
5	Create the onSubmit script "Prevent save if Assigned To missing" on Incident, Active = true. Paste the script from the Appendix and click Submit.
6	Create the onCellEdit script "Prevent state change via list edit" on Incident, field State, Active = true. Paste the script from the Appendix and click Submit.

Environment variables are not used in this project.

5. Folder Structure

ServiceNow stores this work as configuration records, not as files in folders. The structure below shows how the records are organised.

Client (runs in the browser):

```
Incident form and list
|-- UI Policy: High Impact Control
|     |-- Condition: Impact is 1 - High (Reverse if false = true)
|     |-- Action: Assignment group (mandatory)
|     \-- Action: Urgency (read-only)
|-- Client Script (onChange): Auto set urgency for high impact
|-- Client Script (onSubmit): Prevent save if Assigned To missing
\-- Client Script (onCellEdit): Prevent state change via list edit
```

Server (ServiceNow platform):

```
ServiceNow instance
|-- Configuration records: UI Policy, UI Policy Actions, Client Scripts
\-- Incident table: Incident records
```

6. Running the Application

There is no npm command to run. The controls are active as soon as the records are submitted and marked Active.

- **Form:** go to Incident > Create New (or open an existing Incident) to see the UI Policy, onChange and onSubmit behaviour.

- **List:** go to Incident > All and double-click the State field to see the onCellEdit behaviour.

7. API Documentation

The project does not expose any REST endpoints. It uses the ServiceNow client-side scripting API (g_form) inside the Client Scripts. The calls used are documented below.

Method	Parameters	Example in this project
g_form.getValue(fieldName)	Field name; returns the field value	g_form.getValue('impact') == '1' checks for High impact
g_form.setValue(fieldName, value)	Field name and the new value	g_form.setValue('urgency', '1') sets Urgency to High
g_form.addInfoMessage(message)	Message text	Shows "Urgency set to High for High impact incident."
g_form.showErrorBox(fieldName, message)	Field name and error text	Shows the Assigned To error on the form
onSubmit return value	return false stops the save, return true allows it	Returns false when Impact is High and Assigned To is empty
callback(false) in onCellEdit	false cancels the list edit	Blocks the State change after the alert

8. Authentication

Authentication and authorization are handled by the ServiceNow platform. Users log in to the instance with their own account, and the project does not add its own tokens, sessions or login code.

- **Authentication:** the administrator logs in to the instance with administrative or configuration access to create the UI Policy and Client Scripts.
- **Authorization:** only users with this access can create or change the UI Policy and Client Scripts. All users who work on Incidents are then subject to the rules.

9. User Interface

The figures below show the configuration records and the behaviour on the Incident form and list.

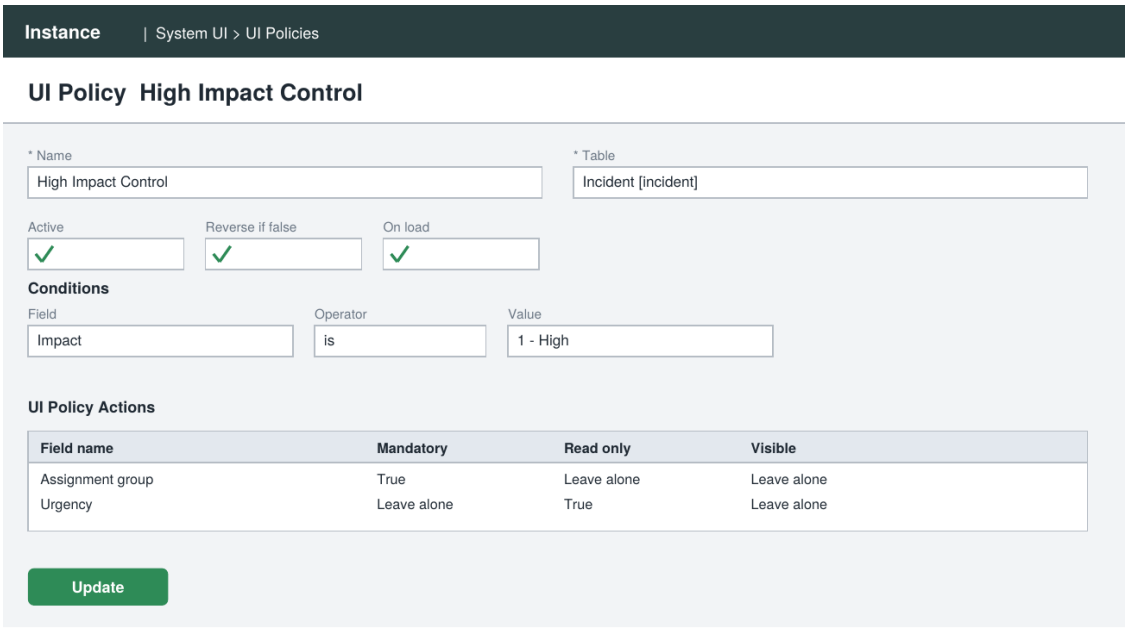


Figure 9.1: UI Policy "High Impact Control" (illustrative mock-up)

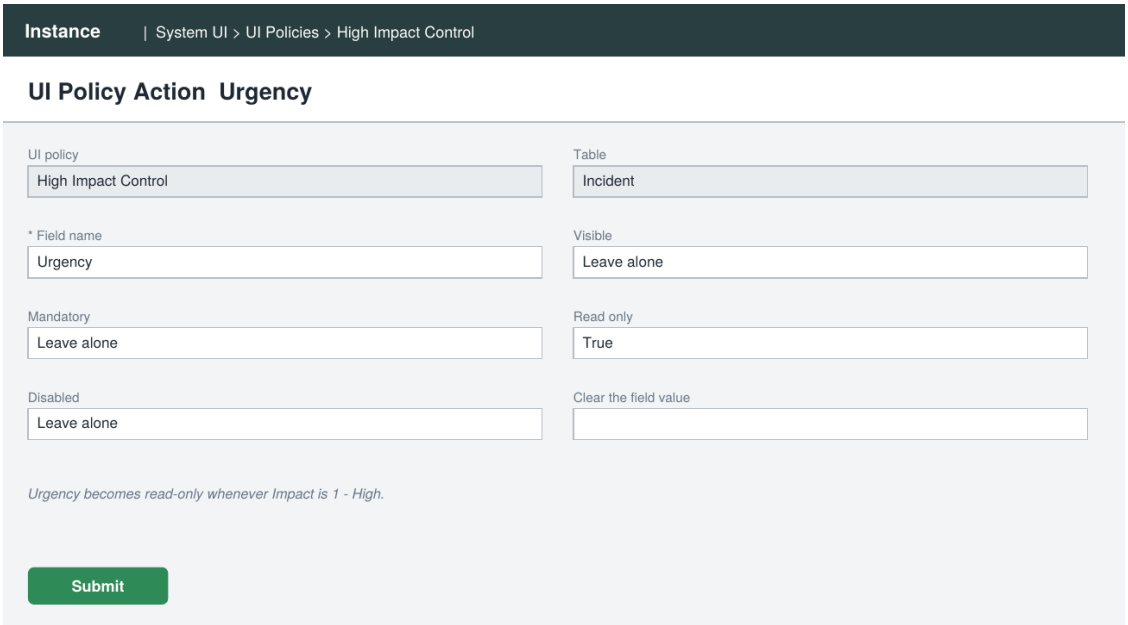


Figure 9.2: UI Policy Action: Urgency read-only (illustrative mock-up)

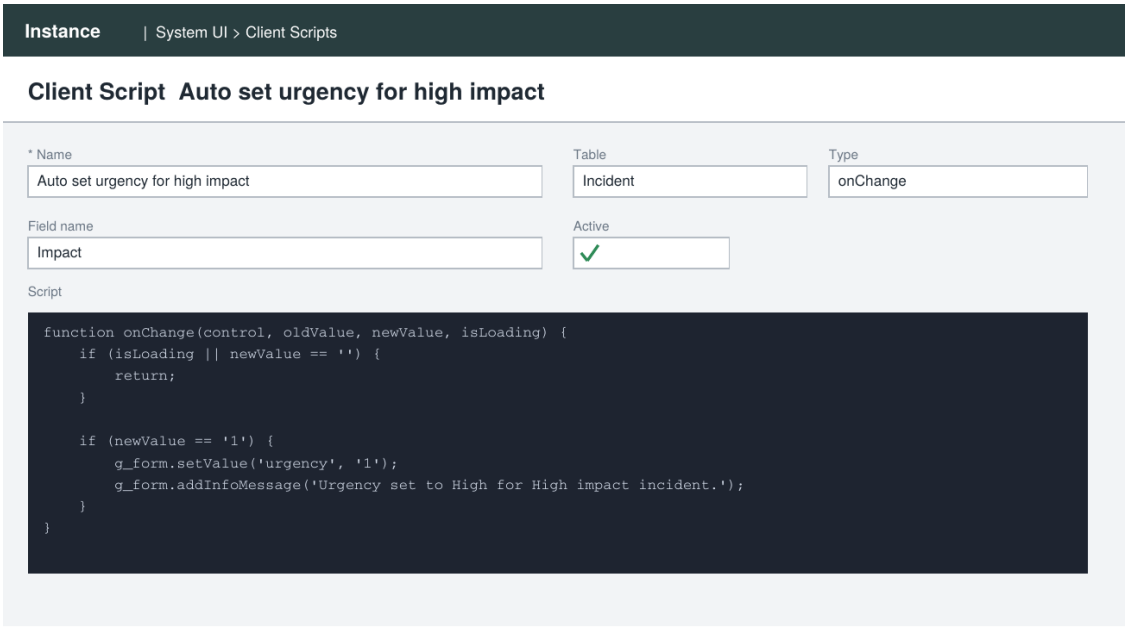


Figure 9.3: onChange Client Script (illustrative mock-up)

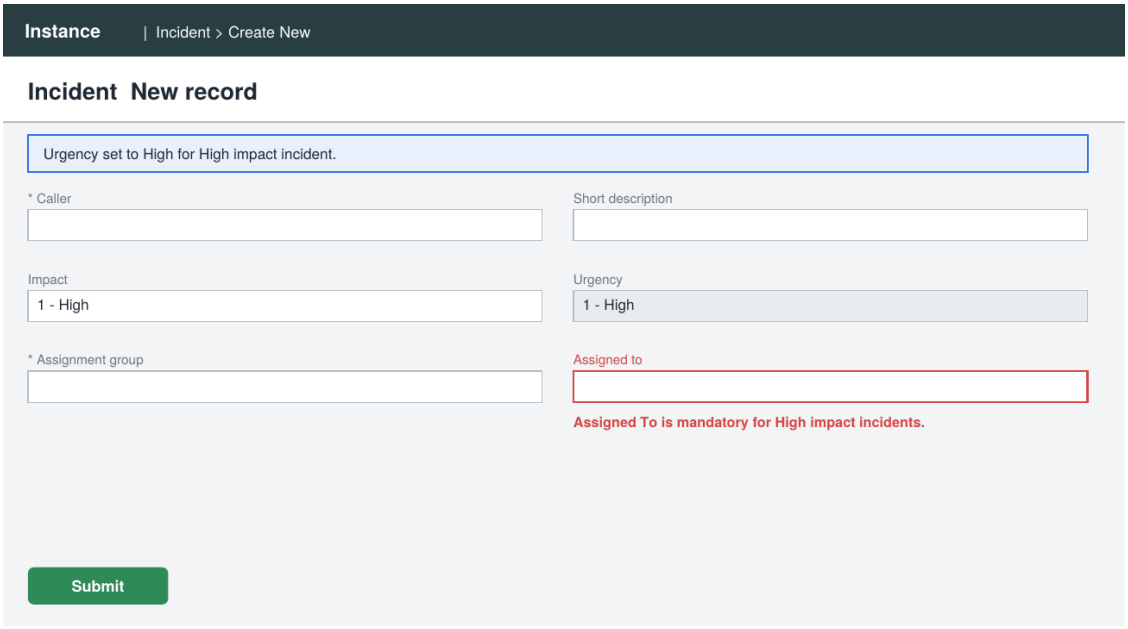


Figure 9.4: onSubmit error on Assigned To (illustrative mock-up)

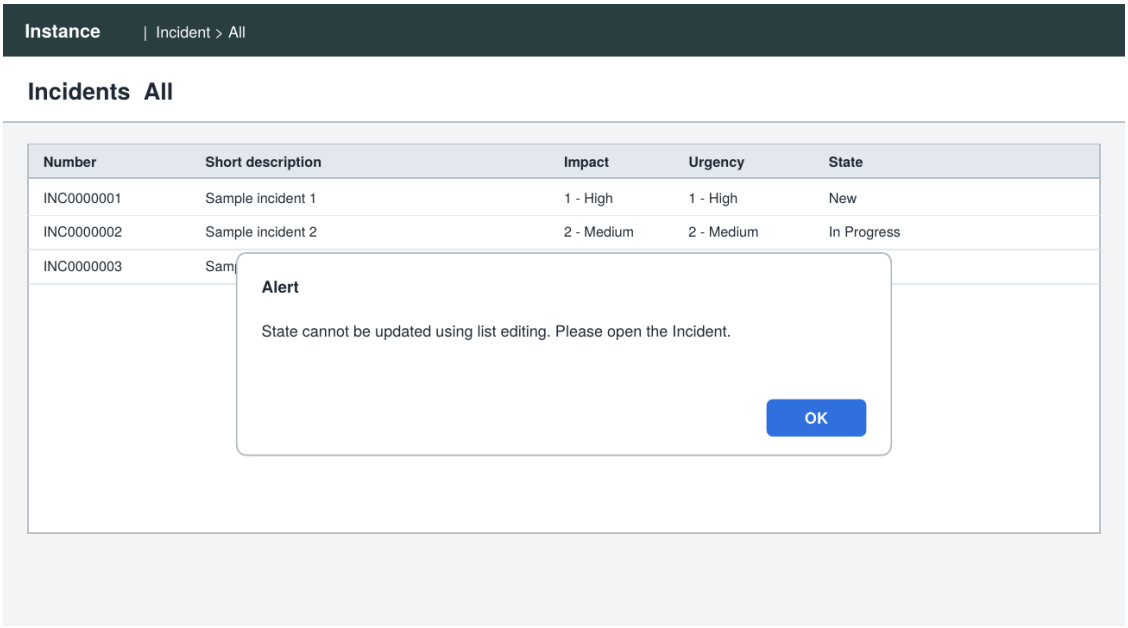


Figure 9.5: onCellEdit alert on list edit (illustrative mock-up)

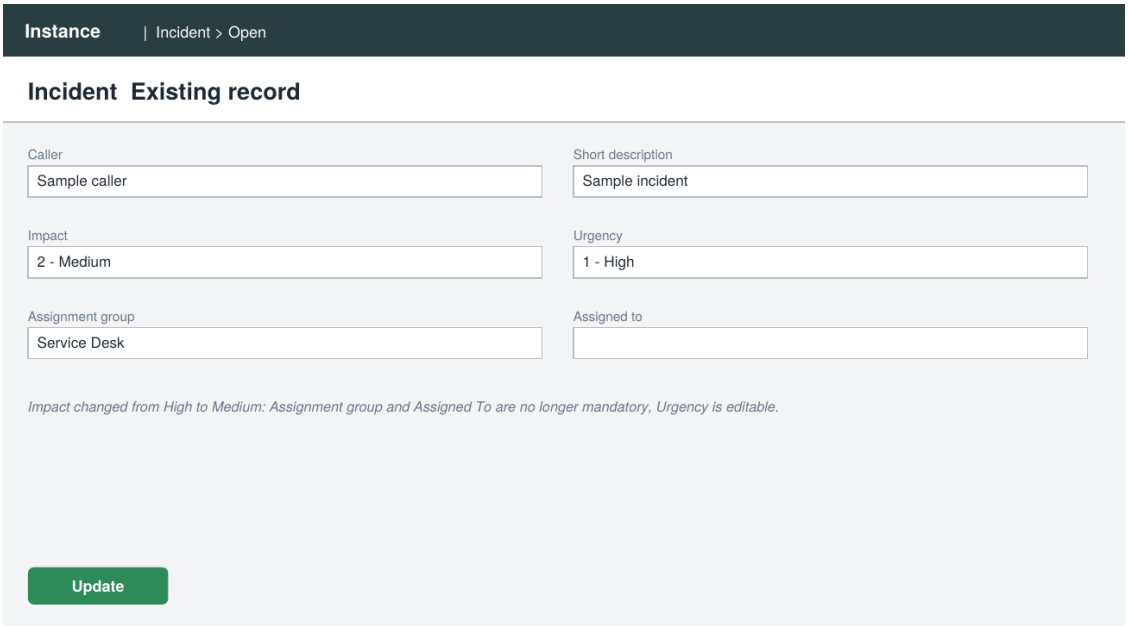


Figure 9.6: Reverse condition test (Impact changed to Medium) (illustrative mock-up)

10. Testing

Testing strategy: the controls are tested manually in the browser on the ServiceNow instance. Each rule is checked when it applies and when it stops applying. The tools used are the Incident form and the Incident list.

No.	Test case	Steps	Expected result	Result
1	Mandatory enforcement	Create a new Incident, set Impact to High, leave Assigned To empty and click Submit	Incident is not saved; an error says Assigned To is mandatory for High impact; Urgency is High and read-only	
2	Successful save	Fill Assigned To and click Submit	Incident saves with no errors; other rules still work	
3	Reverse condition	Change Impact from High to Medium on an existing Incident and save	Assigned To is not mandatory, Urgency is editable and the record saves	
4	List edit blocking	Double-click State in the Incident list	An alert appears and State is unchanged	
5	Form-based update	Change State on the Incident form and click Update	State change is saved; other rules still apply	

11. Screenshots or Demo

The screenshots are shown in Section 9, User Interface.

12. Known Issues

- The rules run only in the browser, so records created in other ways (for example, through integrations) are not checked by these scripts.
- The UI Policy makes Assignment group mandatory, while the onSubmit script checks Assigned To. The two should be aligned so the same field is required in both places.
- The browser alert shown on list edits can interrupt users.

13. Future Enhancements

- Apply the same pattern to other fields and to other tables, such as Problem or Change.
- Add controls for other Impact or Priority levels.
- Add a server-side check as a backup for records created outside the form.
- Show a message explaining why a field is read-only.
- Keep a reusable test checklist for every new rule.

Appendix: Source Code

onChange Client Script: Auto set urgency for high impact

```
function onChange(control, oldValue, newValue, isLoading) {
    if (isLoading || newValue == '') {
        return;
    }

    if (newValue == '1') {
        g_form.setValue('urgency', '1');
        g_form.addInfoMessage('Urgency set to High for High impact incident.');
    }
}
```

onSubmit Client Script: Prevent save if Assigned To missing

```
function onSubmit() {
    if (g_form.getValue('impact') == '1' &&
        g_form.getValue('assigned_to') == '') {

        g_form.showErrorBox(
            'assigned_to',
            'Assigned To is mandatory for High impact incidents.'
        );
        return false;
    }
    return true;
}
```

onCellEdit Client Script: Prevent state change via list edit

```
function onCellEdit(sysIDs, table, oldValues, newValue, callback) {

    alert('State cannot be updated using list editing. Please open the Incident.');

    callback(false);
}
```